ICT133 2024S1 Exam

Feel free to contact me at maths_and_programming@outlook.sg if you need tuition. I provide university level tuition for the following SUSS units:

*	BUS105	Statistics
*	EAS439	Numerical Analysis
*	ENG301	Microprocessor Programming
*	ICT133	Structured Programming
*	ICT162	Object Oriented Programming
•	ICT235	Data Structures and Algorithms
•	ICT325	Algorithm Design and Analysis
•	ICT330	Database Management Systems
*	MTD215	Application of C++ in Multimedia
*	MTD315	Computer Interactive Graphics
*	MTH105	Fundamentals of Mathematics
*	MTH109	Calculus
•	MTH207	Linear Algebra
*	MTH210	Fundamentals of Probability
*	MTH212	Statistical Analysis
•	MTH240	Engineering Mathematics 1
*	MTH316	Multivariable Calculus
*	MTH321	Engineering Mathematics 2
*	MTH355	Basic Mathematical Optimisation
*	MTH357	Applied Regression Analysis I

Donaldson Tan

SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Examination - January Semester 2024

Structured Programming

Thursday, 18 April 2024 **01:00 pm - 03:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions and comprises **6** printed pages (including cover page).
2. You must answer **ALL** questions.
3. You can use ONLY built-in functions and packages covered in the module, e.g., sum, max, min, math, random. You can use ONLY built-in types. Do not write classes.
4. All answers must be written in the answer book.
5. This is **Closed-book** examination.
6. A4 Sized Cheat Sheet are allowed to be brought into exam venue.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT133 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Question 1a

According to the expression of the following python codes, produce the output for line 17 to 19.

```
1  def testSelection(x,y):
2      if x < y and y < 50:
3          print("1st half")
4          if y > 5 and y % 2 == 1:
5              print("A bit odd")
6          elif x % 2 == 1:
7              print("Just odd")
8      else:
9          print("2nd Half")
10         if x > 50 or y < 50:
11             print("Stop")
12         if x % 2 == 0:
13             print("Even")
14         else:
15             print("Not even")
16
17 testSelection(50,50)
18 testSelection(21,31)
19 testSelection(69,96)
```

(7 marks)

Question 1b

Study the given *main()* function below.

```
def main():
    x = 16
    while True:
        if x <= 0:
            break
        print(x)
        x = x - 4

main()
```

What is the output when the *main()* function executes?

Rewrite the above *main()* function, by replacing the while-loop using **ONE (1)** for-loop.

(8 marks)

Question 1c

The following code segment shows a function *isValid( )* that accepts a string as parameter and will return a Boolean value.

Explain in detail, what the function *isValid( )* does?

```
def isValid(x: str) -> bool:

   for n in range(len(x)-1):
      if x[n] > x[n + 1]:
         return False

   return True
```

(5 marks)

Question 2

Read the entire question before attempting. You need to submit each part individually.

Question 2a

Write the following Python function and apply necessary data structures.

getPlayers( ) -> list:

- This function will prompt players to enter their names and returns a list representing TWO unique player names. For example: ['Jack', 'Jim']
- If not, the function will prompt the players to re-enter the names again

<u>Sample run:</u>
Enter player 1 name: Jim
Enter player 2 name: jim
Players' names must be unique!! Please re-enter
Enter player 1 name: Jim
Enter player 2 name: Jack

(10 marks)

Question 2b

Assuming you are given the following functions, and the *getPlayers( )* from Q2(a):

getDeck( ) -> list:

- This function returns a list representing a newly shuffled deck of poker cards in the following format:
  [['Diamond', '10'], ['Spade', '6'], ['Club', '3'], ['Heart', '5'], ……]

drawCard(deck: list) -> list:

- This function randomly selected and returns a card from the given parameter "deck". For example, ['Diamond', '10']
- The selected card will be removed from the deck.

compareCard(card1: list, card2: list) -> bool:

- This function returns True if card1 is bigger than card2, otherwise returns False.

Employing structured programming and using the above given functions, write a *main()* function that allows 2 players to play the following game:

1. At start of the game, obtain **TWO (2)** unique names representing the 2 players.
2. There will be 10 rounds, and a new deck is used.
3. In every round, each player draws a card from the deck.
4. Player with the bigger card will score 1 point.
5. Repeat step 3 & 4, until all 10 rounds are played.
6. Summarize the game result. Some examples:

   Jack 6 : Jim 4
   Jack is the winner for this game!!

   Jason 5 : Jean 5
   Is a draw!!
7. The program then prompts whether the players wish to play.

(20 marks)

Question 3

Question 3a

What is the output produced when execute/apply the sequence of statements?

```
myDict = {1: [5, 2.5], 2: {1: 5, 2: 2.5}, 3: 9}
myList = [5, 2]

print( myDict[1] )
print( myDict[2][2] )
print( myDict[3] * myList[1] )
```

(6 marks)

Question 3b

Trace the given program below and show the content of newDict (line 7) for each iteration of the loop.

```
1 wordList = ['song','world','zoo',
2              'zoom','beater','repeat']
3
4 newDict = {}
5 for word in wordList:
6     newDict[ len(word) ] = word
7     print( newDict )
```

(6 marks)

Question 3c

Write a function *createVowelDictionary(words: list) -> dictionary* to solve the following problem. Read the list of words in the given parameter, and return a dictionary. For each of the word read, find the number of the "vowels" for this word. Use this number as the key to the dictionary, to either build the list or append the word into the list. The returned dictionary should follow the format shown as below:

```
stats = {   3: ['assistants', 'function', 'business'],
            4: ['repeated', 'repeating'],
            2: ['count', 'letters', 'assist'],
            1: ['that', 'word', 'suss']   }
```

Study the program below and code the *createVowelDictionary()* function accordingly:

```python
def createVowelDictionary(words: list):
    # continue your code here...


def main():
    words = [ 'assistants', 'repeated', 'repeating',
              'count', 'function', 'letters', 'business',
              'that', 'word', 'suss', 'assist' ]

    stats = createVowelDictionary( words )
    print( stats )
    """
    # output from above "print( stats )" will be:
    { 3: ['assistants', 'function', 'business'],
      4: ['repeated', 'repeating'],
      2: ['count', 'letters', 'assist'],
      1: ['that', 'word', 'suss'] }
    """

main()
```

(18 marks)

Question 4

Develop an application to solve the following computational problem. The application should process the ballots of 3 candidates in an election. The ballots are stored in a "ballots.txt" file and each line represents a vote for the candidate.

Below is a sample of a "ballots.txt" file:

Liang
Song
Tham
Song
Tham
…
…
…
Liang
Tham
Tham
Song
Tham
Liang

Write Python functions, such that the following Python application can execute.

```python
def main():
    ballotData = readBallotData("ballots.txt")
    winner = getElectionWinner(ballotData)
    print(winner)
    writeBallotResult("results.txt", ballotData)
```

Question 4a

readBallotData(filename: str) -> dict:

- Given a filename as parameter, the function reads the ballots data from the file and populate a dictionary, with candidates' name as the key and number of votes receives as the value.

The dictionary assigned to ballotData at line 2 may look like the followings:

{'Liang': 10, 'Song': 13, 'Tham': 18}

(8 marks)

Question 4b

getElectionWinner(ballotData: dict) -> str:

- This function has ONE parameter: ballotData, which is the dictionary populated from function readBallotData ( )
- This function will return the name of the candidate that received the most votes (assuming there is no tie).
- Based on the ballotData shown in Q4(a), the winner is 'Tham'.

(6 marks)

Question 4c

writeBallotResult (filename: str, ballotData: dict) -> None:

- Given the election data in parameter "ballotData", this function writes the election result to the filename provided.
- The file will starts with a header line: "Name Votes".
- Subsequent lines will contain the candidate's name and votes received.
- The winner is printed on the last line of the file.
- Sample output file:

```
Name    Votes
Liang   100,234
Song    132,788
Tham    589,333

Winner is Tham
```

(6 marks)

----- END OF PAPER -----

1(a) 2nd Half
Even
for half
a bit odd
2nd Half
Stop

(b) output: 16
12
8
4

```
def main():
    for i in range(16, 0, -4):
        print(i)
```

(c) The function isInvalid() evaluates if a string is in increasing order.

2(a)

```
def getPlayers() -> List[str]:
    names = []
    unique = []
    while len(names) < 2:
        name = input(f"Please enter Player {len(names)+1} name:")
        if len(names) == 0:
            names.append(name)
            unique.append(name.lower())
        elif name.lower() in unique:
            print("Players' names must be unique. Please re-enter")
            names = []
            unique = []
        else:
            names.append(name)
            unique.append(name.lower())
    return names
```

2(b)

```
def main():
    names = getPlayers()
    scores = [0, 0]
    deck = getDeck()
    nextGame = True
    while nextGame:
        nextGame = False
        for _ in range(10):
            draw0 = drawCard(deck)
            draw1 = drawCard(deck)
            if compareCard(draw0, draw1):
                scores[0] += 1
            elif compareCard(draw1, draw0):
                scores[1] += 1
        print(f"{names[0]} {scores[0]} : {names[1]} {scores[1]}")
        if scores[0] > scores[1]:
            print(f"{names[0]} is the winner for this game!")
        elif score[1] > score[0]:
            print(f"{names[1]} is the winner for this game!")
        else:
            print("Is a draw")
        Y/N = input("Do you want to play another game? [Y/N]").upper()
        nextGame = Y/N == 'Y'
```

3(a) [5, 2.5]
2.5
18

(b)

iteration	new Dict
0	{ 4: "song" }
1	{ 4: "song", 5: "world" }
2	{ 3: "zoo", 4: "song", 5: "world" }
3	{ 3: "zoo", 4: "zoom", 5: "world" }
4	{ 3: "zoo", 4: "zoom", 5: "world", 6: "beator" }
5	{ 3: "zoo", 4: "zoom", 5: "world", 6: "repeat" }

(c)

```
def create Vowel Dictionary (words: List[Str]) -> Dict[int, List[str]]:
    stats = { }
    for word in words:
        count = 0
        for c in word:
            if c in ['a', 'e', 'i', 'o', 'u']:
                count += 1
        if count in stats:
            stats[count].append(word)
        else:
            stats[count] = [word]
    return stats
```

4(a)

```
def readBallotData(filename: str) -> Dict[str, int]:
    votes = {}
    with open(filename) as f:
        for line in f:
            candidate = line.strip()
            if candidate in votes:
                votes[candidate] += 1
            else:
                votes[candidate] = 1
    return votes
```

(b)

```
def getElectionWinner(ballotData: Dict[str, int]) -> str:
    winner = " "
    maxCount = 0
    for candidate in ballotData:
        if ballotData[candidate] > maxCount:
            winner = candidate
            maxCount = ballotData[candidate]
    return winner
```

(c)

```
def writeBallotResult(filename: str, ballotData: Dict[str, int]) -> None:
    width = max([len(candidate) for candidate in ballotData]) + 1
    print(f"{'Name': {width}} Votes")
    for candidate in ballotData:
        print(f"{candidate.ljust(width)} {ballotData[candidate]}")
    print(f"Winner is {getElectionWinner(ballotData)}")
```